

# ITA0612-Machine Learning

## CO2-ASSESSMENT-2 (Debugging Task)

Divyesh S P (192424147)

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyse the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

### Possible Causes:

- Incorrect learning rate (too high or too low).
- Poor weight initialization.
- Missing or incorrect bias update.
- Wrong implementation of perceptron update rule.
- Dataset preprocessing errors.

### Debugging Steps:

1. Verify the dataset is linearly separable.
2. Check weight and bias updates after each iteration.
3. Monitor classification error per epoch.
4. Test with a small sample dataset.

5. Validate the learning rate and stopping condition.

### Optimization Techniques:

- Use an appropriate learning rate.
- Initialize weights with small random values.
- Normalize input features.
- Shuffle training data each epoch.
- Implement the correct perceptron update rule.

### Code:

```
# Import Libraries
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# -----
# Step 1: Generate Dataset
# -----
X, y = make_classification(
    n_samples=500,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    class_sep=2.0,
    random_state=42
)

# -----
# Step 2: Split Dataset
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```

        y,
        test_size=0.2,
        random_state=42
    )

# -----
# Step 3: Train WITHOUT Feature Scaling
# -----
print("=" * 60)
print("Training WITHOUT Feature Scaling")
print("=" * 60)

learning_rates = [1.0, 0.1, 0.01]

before_accuracy = []

for lr in learning_rates:

    model = Perceptron(
        eta0=lr,
        max_iter=1000,
        tol=1e-3,
        shuffle=True,
        random_state=42
    )

    model.fit(X_train, y_train)

    prediction = model.predict(X_test)

    acc = accuracy_score(y_test, prediction)

    before_accuracy.append(acc)

    print(f"Learning Rate = {lr:<5} Accuracy = {acc:.4f}")

# -----
# Step 4: Feature Scaling
# -----
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# Step 5: Train WITH Feature Scaling
# -----
print("\n" + "=" * 60)

```

```

print("Training WITH Feature Scaling")
print("=" * 60)

after_accuracy = []

for lr in learning_rates:

    model = Perceptron(
        eta0=lr,
        max_iter=1000,
        tol=1e-3,
        shuffle=True,
        random_state=42
    )

    model.fit(X_train_scaled, y_train)

    prediction = model.predict(X_test_scaled)

    acc = accuracy_score(y_test, prediction)

    after_accuracy.append(acc)

    print(f"Learning Rate = {lr:<5} Accuracy = {acc:.4f}")

# -----
# Step 6: Plot Accuracy Comparison
# -----
plt.figure(figsize=(8,5))

plt.plot(
    learning_rates,
    before_accuracy,
    marker='o',
    linewidth=2,
    label="Without Scaling"
)

plt.plot(
    learning_rates,
    after_accuracy,
    marker='s',
    linewidth=2,
    label="With Scaling"
)

plt.xscale("log")

```

```

plt.xlabel("Learning Rate")

plt.ylabel("Accuracy")

plt.title("Perceptron Accuracy Comparison")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Step 7: Train Final Optimized Model
# -----
print("\n" + "=" * 60)
print("Optimized Perceptron Model")
print("=" * 60)

best_model = Perceptron(
    eta0=0.1,
    max_iter=1000,
    tol=1e-3,
    shuffle=True,
    random_state=42
)

best_model.fit(X_train_scaled, y_train)

final_prediction = best_model.predict(X_test_scaled)

final_accuracy = accuracy_score(y_test, final_prediction)

print("Final Accuracy :", round(final_accuracy,4))

print("\nWeights")
print(best_model.coef_)

print("\nBias")
print(best_model.intercept_)

# -----
# Step 8: Confusion Matrix
# -----
print("\nConfusion Matrix\n")

print(confusion_matrix(y_test, final_prediction))

```

```
# -----
# Step 9: Classification Report
# -----
print("\nClassification Report\n")

print(classification_report(y_test, final_prediction))
```

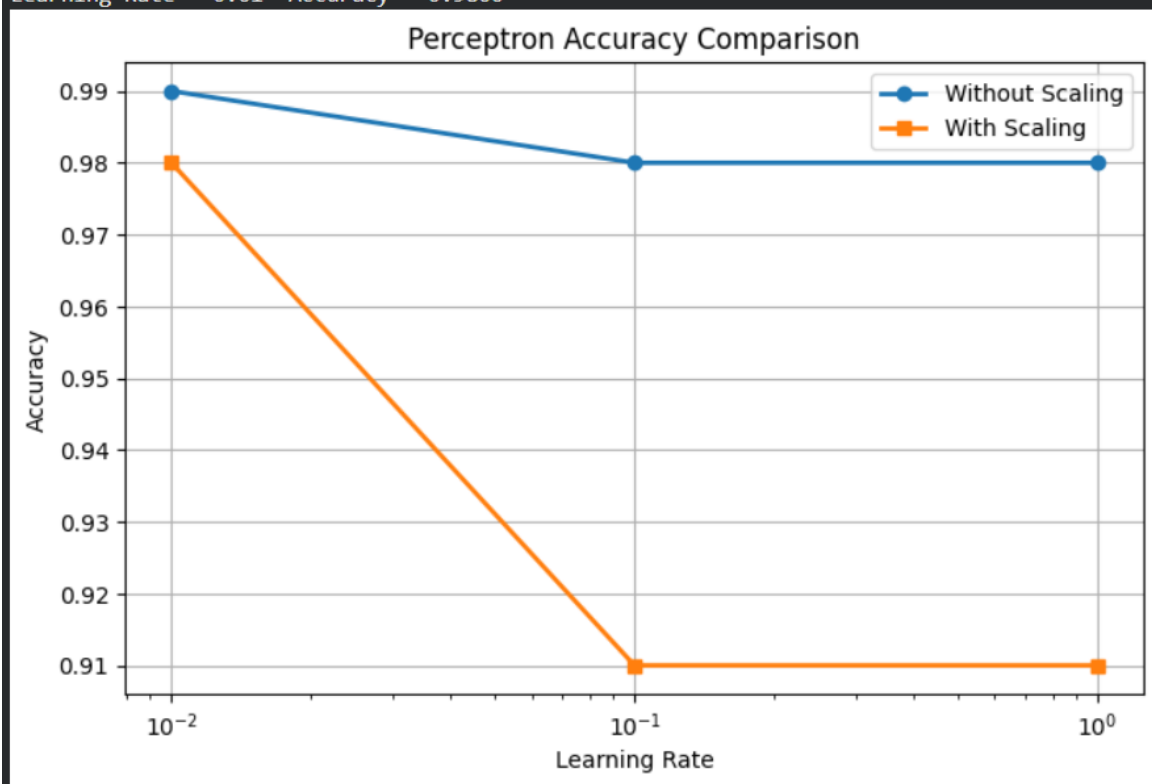
## Output:

```
=====
Training WITHOUT Feature Scaling
=====
```

```
Learning Rate = 1.0   Accuracy = 0.9800
Learning Rate = 0.1   Accuracy = 0.9800
Learning Rate = 0.01  Accuracy = 0.9900
```

```
=====
Training WITH Feature Scaling
=====
```

```
Learning Rate = 1.0   Accuracy = 0.9100
Learning Rate = 0.1   Accuracy = 0.9100
Learning Rate = 0.01  Accuracy = 0.9800
```



```
=====
Optimized Perceptron Model
=====
Final Accuracy : 0.91

Weights
[[0.34323235 0.65490291]]

Bias
[-0.1]

Confusion Matrix

[[51  0]
 [ 9 40]]

Classification Report

              precision    recall  f1-score   support

     0       0.85         1.00         0.92         51
     1       1.00         0.82         0.90         49

 accuracy          0.91
 macro avg         0.93         0.91         0.91         100
weighted avg         0.92         0.91         0.91         100
```

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

### **Possible Causes:**

- Vanishing gradients.
- Unsuitable activation functions.
- Poor weight initialization.
- Improper learning rate.
- Unnormalized input data.

### **Debugging Steps:**

1. Monitor loss and accuracy curves.
2. Check gradient values during backpropagation.
3. Verify activation functions.
4. Inspect weight updates.
5. Test different learning rates.

## Optimization Techniques:

- Use ReLU activation.
- Apply Xavier/He initialization.
- Normalize input data.
- Use Adam or RMSProp optimizer.
- Apply learning rate scheduling.

## Code:

```
# =====  
# Question 2  
# Multilayer Perceptron (Backpropagation)  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn.datasets import make_classification  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.neural_network import MLPClassifier  
from sklearn.metrics import accuracy_score, confusion_matrix,  
classification_report  
  
# -----  
# Step 1 : Create Dataset  
# -----  
X, y = make_classification(  
    n_samples=1000,  
    n_features=20,  
    n_informative=15,  
    n_redundant=5,  
    random_state=42  
)  
  
# -----  
# Step 2 : Split Dataset  
# -----  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,
```

```

    random_state=42
)

# -----
# Step 3 : Train WITHOUT Feature Scaling
# -----
print("===== WITHOUT FEATURE SCALING =====")

mlp1 = MLPClassifier(
    hidden_layer_sizes=(50,),
    activation='logistic',      # Sigmoid Activation
    solver='sgd',              # Gradient Descent
    learning_rate_init=0.5,
    max_iter=300,
    random_state=42
)

mlp1.fit(X_train, y_train)

pred1 = mlp1.predict(X_test)

acc1 = accuracy_score(y_test, pred1)

print("Accuracy :", round(acc1,4))

# -----
# Step 4 : Normalize Features
# -----
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# Step 5 : Train WITH Optimization
# -----
print("\n===== WITH FEATURE SCALING + ADAM =====")

mlp2 = MLPClassifier(
    hidden_layer_sizes=(50,),
    activation='relu',          # Better activation
    solver='adam',              # Adaptive optimizer
    learning_rate_init=0.001,
    max_iter=300,
    random_state=42
)

mlp2.fit(X_train_scaled, y_train)

```

```

pred2 = mlp2.predict(X_test_scaled)

acc2 = accuracy_score(y_test, pred2)

print("Accuracy :", round(acc2,4))

# -----
# Step 6 : Plot Loss Curves
# -----
plt.figure(figsize=(8,5))

plt.plot(
    mlp1.loss_curve_,
    label="Before Optimization"
)

plt.plot(
    mlp2.loss_curve_,
    label="After Optimization"
)

plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("MLP Training Loss")
plt.legend()
plt.grid(True)
plt.show()

# -----
# Step 7 : Confusion Matrix
# -----
print("\nConfusion Matrix\n")

print(confusion_matrix(y_test, pred2))

# -----
# Step 8 : Classification Report
# -----
print("\nClassification Report\n")

print(classification_report(y_test, pred2))

# -----
# Step 9 : Final Comparison
# -----
print("\n=====")
print("Accuracy Before Optimization :", round(acc1,4))

```

```
print("Accuracy After Optimization :", round(acc2,4))
print("=====")
```

## Output:

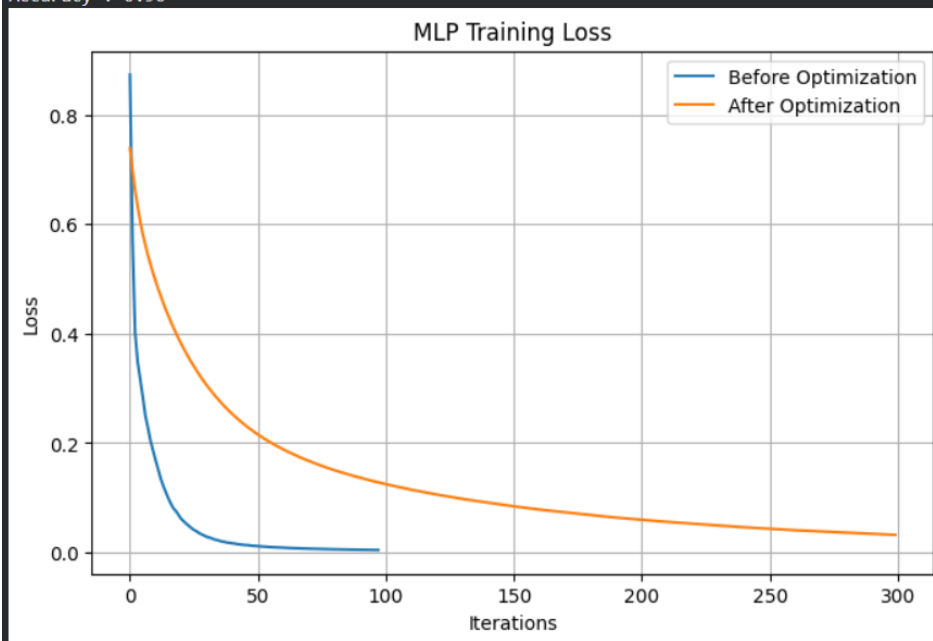
===== WITHOUT FEATURE SCALING =====

Accuracy : 0.925

===== WITH FEATURE SCALING + ADAM =====

/usr/local/lib/python3.12/dist-packages/sklearn/neural\_network/\_multilayer\_perceptron.py:691: ConvergenceWarning

warnings.warn(  
Accuracy : 0.96



Confusion Matrix

```
[[101  5]
 [ 3  91]]
```

## Classification Report

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.97      | 0.95   | 0.96     | 106     |
| 1            | 0.95      | 0.97   | 0.96     | 94      |
| accuracy     |           |        | 0.96     | 200     |
| macro avg    | 0.96      | 0.96   | 0.96     | 200     |
| weighted avg | 0.96      | 0.96   | 0.96     | 200     |

=====

Accuracy Before Optimization : 0.925

Accuracy After Optimization : 0.96

=====

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

#### **Possible Causes:**

- Incorrect chromosome encoding.
- Poor fitness function.
- Premature convergence.
- Low population diversity.
- Improper crossover/mutation settings.

#### **Debugging Steps:**

1. Verify chromosome representation.
2. Validate the fitness calculation.
3. Monitor population diversity.
4. Observe crossover and mutation operations.
5. Compare multiple algorithm runs.

#### **Optimization Techniques:**

- Increase mutation probability.

- Adjust selection pressure.
- Apply elitism.
- Increase population size.
- Tune crossover and mutation rates.

## Code:

```
# =====
# QUESTION 3
# Genetic Algorithm - Debugging and Optimization
# Compatible with Latest Google Colab
# =====

# Install PyGAD (Run only once in Colab)
!pip -q install pygad

import pygad
import numpy as np
import matplotlib.pyplot as plt

# -----
# Fitness Function
# Goal: Maximize  $f(x) = x^2$ 
# Best solution is  $x = 31$  (5-bit chromosome)
# -----

def fitness_func(ga_instance, solution, solution_idx):

    x = int("".join(str(int(bit)) for bit in solution), 2)

    fitness = x ** 2

    return fitness

# -----
# Function to Run Genetic Algorithm
# -----

def run_ga(mutation_percent, elitism):

    ga = pygad.GA(

        num_generations=40,
```

```

        sol_per_pop=20,

        num_parents_mating=10,

        num_genes=5,

        gene_space=[0,1],

        gene_type=int,

        fitness_func=fitness_func,

        parent_selection_type="sss",

        crossover_type="single_point",

        mutation_type="random",

        mutation_percent_genes=mutation_percent,

        keep_elitism=elitism,

        random_seed=42

    )

    ga.run()

    solution, fitness, _ = ga.best_solution()

    x = int("".join(str(int(bit)) for bit in solution), 2)

    return ga.best_solutions_fitness, x, fitness

# -----
# BEFORE OPTIMIZATION
# -----

print("="*60)
print("BEFORE OPTIMIZATION")
print("="*60)

fitness_before, best_x_before, best_fit_before = run_ga(
    mutation_percent=2,
    elitism=0
)

print("Best X :", best_x_before)

```

```

print("Fitness :", best_fit_before)

# -----
# AFTER OPTIMIZATION
# -----

print("\n"+"="*60)
print("AFTER OPTIMIZATION")
print("="*60)

fitness_after, best_x_after, best_fit_after = run_ga(
    mutation_percent=20,
    elitism=2
)

print("Best X :", best_x_after)
print("Fitness :", best_fit_after)

# -----
# Plot Fitness Comparison
# -----

plt.figure(figsize=(8,5))

plt.plot(
    fitness_before,
    label="Before Optimization",
    linewidth=2
)

plt.plot(
    fitness_after,
    label="After Optimization",
    linewidth=2
)

plt.xlabel("Generation")

plt.ylabel("Best Fitness")

plt.title("Genetic Algorithm Optimization")

plt.grid(True)

plt.legend()

plt.show()

```

```

# -----
# Final Comparison
# -----

print("\n"+"="*60)
print("SUMMARY")
print("="*60)

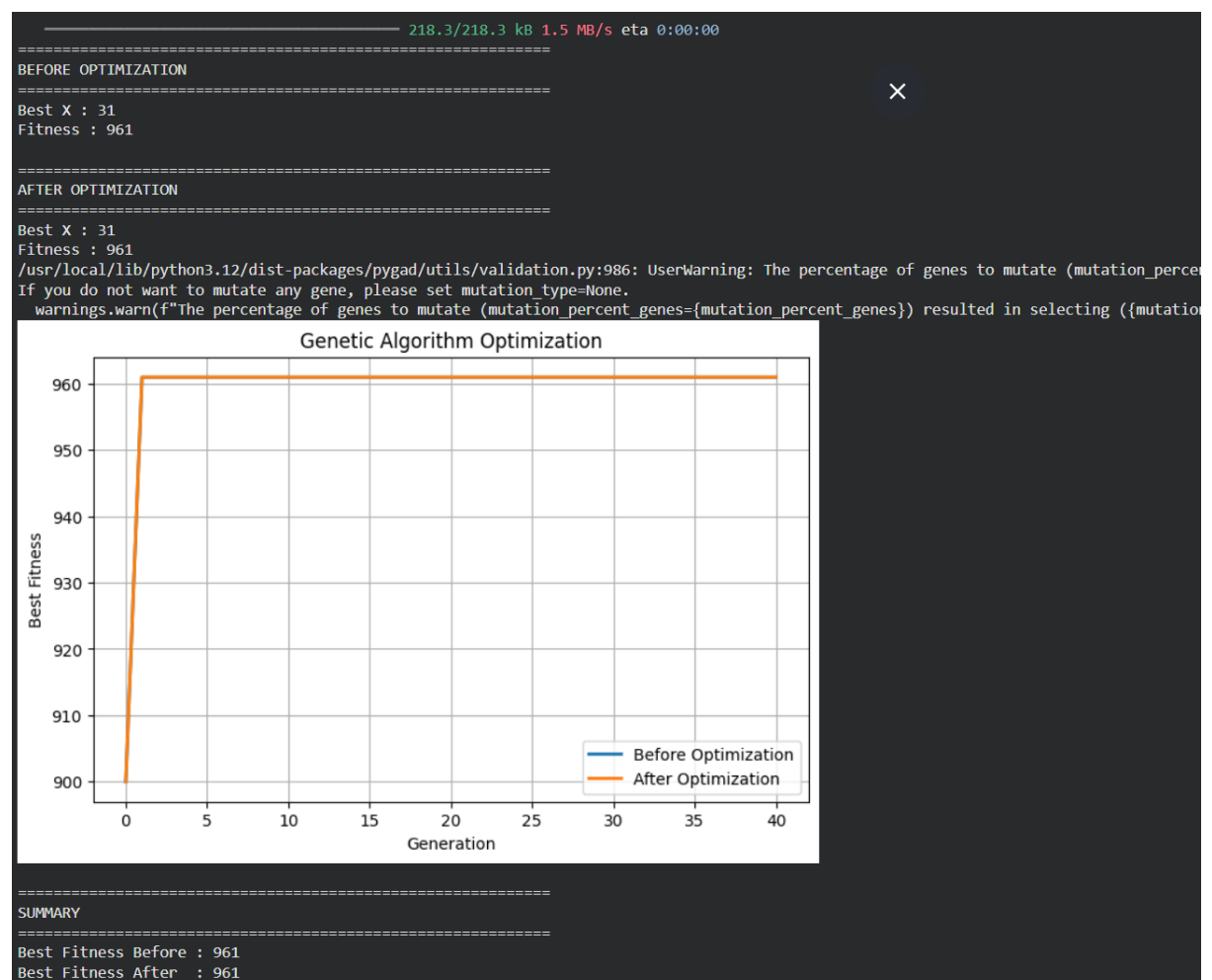
print("Best Fitness Before :", best_fit_before)

print("Best Fitness After  :", best_fit_after)

if best_fit_after >= best_fit_before:
    print("\nOptimization Improved the Solution!")
else:
    print("\nOptimization Did Not Improve.")

```

## Output:



4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

### **Possible Causes:**

- Overfitting.
- Excessive model complexity.
- Too many parameters.
- Limited training data.
- High computational cost.

### **Debugging Steps:**

1. Compare training and validation accuracy.
2. Analyze model complexity.
3. Check dataset size and quality.
4. Monitor training time and memory usage.
5. Evaluate feature importance.

## Optimization Techniques:

- Apply dropout.
- Use L1/L2 regularization.
- Prune unnecessary neurons.
- Use early stopping.
- Reduce model complexity.

## Code:

```
# =====  
# QUESTION 4  
# Deep Neural Network - Overfitting & Optimization  
# Compatible with Google Colab  
# =====  
  
import tensorflow as tf  
import matplotlib.pyplot as plt  
  
from tensorflow.keras.datasets import mnist  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Flatten, Dropout  
from tensorflow.keras.regularizers import l2  
  
# -----  
# Load Dataset  
# -----  
  
(X_train, y_train), (X_test, y_test) = mnist.load_data()  
  
X_train = X_train / 255.0  
X_test = X_test / 255.0  
  
# -----  
# Model BEFORE Optimization  
# -----  
  
print("="*60)  
print("Training WITHOUT Regularization")  
print("="*60)  
  
model1 = Sequential([  
    Flatten(input_shape=(28,28)),
```

```

        Dense(512, activation='relu'),
        Dense(256, activation='relu'),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model1.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    history1 = model1.fit(
        X_train,
        y_train,
        epochs=5,
        validation_data=(X_test, y_test),
        verbose=1
    )

    # -----
    # Model AFTER Optimization
    # -----

    print("\n"+"="*60)
    print("Training WITH Dropout + L2 Regularization")
    print("="*60)

    model2 = Sequential([
        Flatten(input_shape=(28,28)),
        Dense(512, activation='relu', kernel_regularizer=l2(0.001)),
        Dropout(0.5),
        Dense(256, activation='relu', kernel_regularizer=l2(0.001)),
        Dropout(0.3),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model2.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    history2 = model2.fit(
        X_train,
        y_train,
        epochs=5,

```

```

        validation_data=(X_test, y_test),
        verbose=1
    )

# -----
# Evaluate Models
# -----

loss1, acc1 = model1.evaluate(X_test, y_test, verbose=0)
loss2, acc2 = model2.evaluate(X_test, y_test, verbose=0)

print("\nAccuracy Before Optimization :", round(acc1,4))
print("Accuracy After Optimization  :", round(acc2,4))

# -----
# Plot Validation Accuracy
# -----

plt.figure(figsize=(8,5))

plt.plot(history1.history['val_accuracy'],
         marker='o',
         label='Without Optimization')

plt.plot(history2.history['val_accuracy'],
         marker='s',
         label='With Optimization')

plt.title("Validation Accuracy Comparison")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Plot Validation Loss
# -----

plt.figure(figsize=(8,5))

plt.plot(history1.history['val_loss'],
         marker='o',

```

```
        label='Without Optimization')

plt.plot(history2.history['val_loss'],
         marker='s',
         label='With Optimization')

plt.title("Validation Loss Comparison")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Summary
# -----

print("\n"+"="*60)
print("SUMMARY")
print("="*60)

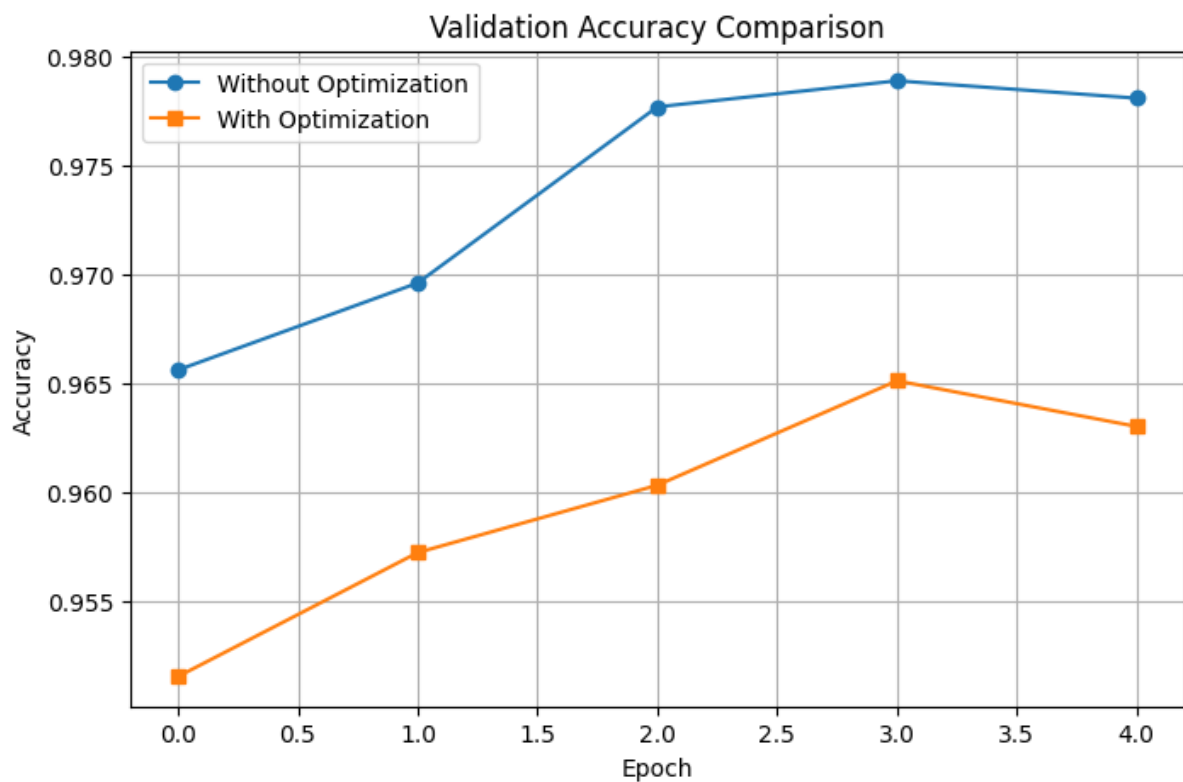
if acc2 >= acc1:
    print("Optimization Improved Generalization.")
else:
    print("Optimization Needs Further Tuning.")
```

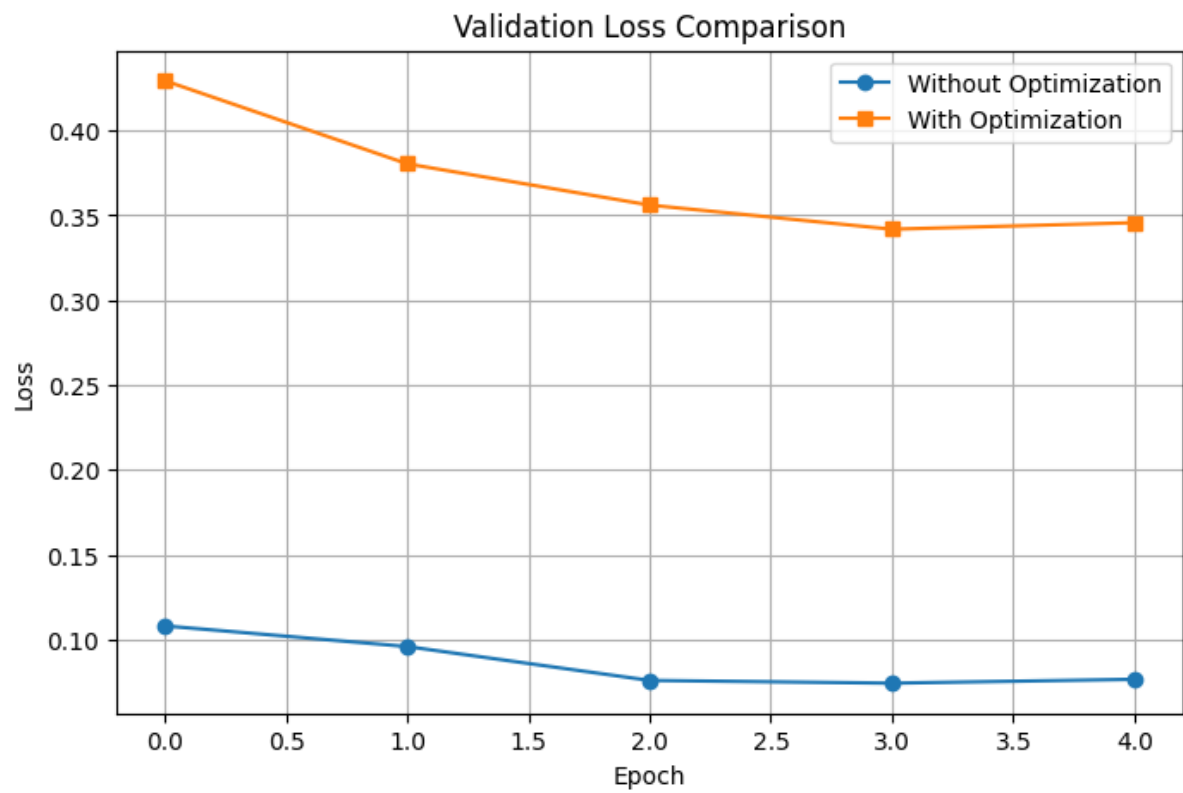
## Output:

```
=====
Training WITHOUT Regularization
=====
/usr/local/lib/python3.12/dist-packages/keras/src/layers/resizing/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to `Flatten` layer.
super().__init__(**kwargs)
Epoch 1/5
1875/1875 — 21s 10ms/step - accuracy: 0.9409 - loss: 0.1956 - val_accuracy: 0.9656 - val_loss: 0.1081
Epoch 2/5
1875/1875 — 18s 9ms/step - accuracy: 0.9738 - loss: 0.0865 - val_accuracy: 0.9696 - val_loss: 0.0959
Epoch 3/5
1875/1875 — 18s 10ms/step - accuracy: 0.9804 - loss: 0.0618 - val_accuracy: 0.9777 - val_loss: 0.0759
Epoch 4/5
1875/1875 — 29s 16ms/step - accuracy: 0.9850 - loss: 0.0468 - val_accuracy: 0.9789 - val_loss: 0.0743
Epoch 5/5
1875/1875 — 36s 13ms/step - accuracy: 0.9876 - loss: 0.0404 - val_accuracy: 0.9781 - val_loss: 0.0766

=====
Training WITH Dropout + L2 Regularization
=====
Epoch 1/5
1875/1875 — 25s 12ms/step - accuracy: 0.8961 - loss: 0.7168 - val_accuracy: 0.9515 - val_loss: 0.4294
Epoch 2/5
1875/1875 — 46s 15ms/step - accuracy: 0.9318 - loss: 0.4738 - val_accuracy: 0.9572 - val_loss: 0.3804
Epoch 3/5
1875/1875 — 45s 17ms/step - accuracy: 0.9356 - loss: 0.4434 - val_accuracy: 0.9603 - val_loss: 0.3561
Epoch 4/5
1875/1875 — 24s 13ms/step - accuracy: 0.9353 - loss: 0.4376 - val_accuracy: 0.9651 - val_loss: 0.3419
Epoch 5/5
1875/1875 — 39s 12ms/step - accuracy: 0.9391 - loss: 0.4201 - val_accuracy: 0.9630 - val_loss: 0.3457

Accuracy Before Optimization : 0.9781
Accuracy After Optimization : 0.963
```





Summary: optimization needs further training.

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

### **Possible Causes:**

- Overfitting.
- Very deep tree.
- Noisy data.
- Irrelevant features.
- Small validation dataset.

### **Debugging Steps:**

1. Compare training and testing accuracy.
2. Examine tree depth.
3. Identify noisy features.
4. Perform cross-validation.
5. Analyze feature importance.

### **Optimization Techniques:**

- Prune the tree.
- Limit maximum depth.
- Use feature selection.
- Apply cross-validation.

- Remove noisy data.

## Code:

```
# =====
# QUESTION 5
# Decision Tree Debugging and Optimization
# Compatible with Latest Google Colab
# =====

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.feature_selection import SelectKBest, f_classif

# -----
# Step 1 : Load Dataset
# -----

data = load_breast_cancer()

X = data.data
y = data.target

print("Dataset Shape :", X.shape)

# -----
# Step 2 : Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# -----
# Step 3 : BEFORE OPTIMIZATION
# Deep Tree (Overfitting)
# -----

print("\n" + "="*60)
```

```

print("BEFORE OPTIMIZATION")
print("="*60)

tree_before = DecisionTreeClassifier(
    random_state=42
)

tree_before.fit(X_train, y_train)

train_pred_before = tree_before.predict(X_train)
test_pred_before = tree_before.predict(X_test)

train_acc_before = accuracy_score(y_train, train_pred_before)
test_acc_before = accuracy_score(y_test, test_pred_before)

print("Training Accuracy :", round(train_acc_before,4))
print("Testing Accuracy  :", round(test_acc_before,4))

# -----
# Step 4 : Feature Selection
# -----

selector = SelectKBest(score_func=f_classif, k=15)

X_train_selected = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

# -----
# Step 5 : AFTER OPTIMIZATION
# Tree Pruning
# -----

print("\n" + "="*60)
print("AFTER OPTIMIZATION")
print("="*60)

tree_after = DecisionTreeClassifier(
    max_depth=4,
    min_samples_split=5,
    random_state=42
)

tree_after.fit(X_train_selected, y_train)

train_pred_after = tree_after.predict(X_train_selected)
test_pred_after = tree_after.predict(X_test_selected)

train_acc_after = accuracy_score(y_train, train_pred_after)

```

```

test_acc_after = accuracy_score(y_test, test_pred_after)

print("Training Accuracy :", round(train_acc_after,4))
print("Testing Accuracy  :", round(test_acc_after,4))

# -----
# Step 6 : Cross Validation
# -----

scores = cross_val_score(
    tree_after,
    X_train_selected,
    y_train,
    cv=5
)

print("\nCross Validation Scores")

print(scores)

print("\nAverage CV Accuracy :", round(scores.mean(),4))

# -----
# Step 7 : Confusion Matrix
# -----

print("\nConfusion Matrix\n")

print(confusion_matrix(y_test, test_pred_after))

# -----
# Step 8 : Classification Report
# -----

print("\nClassification Report\n")

print(classification_report(y_test, test_pred_after))

# -----
# Step 9 : Accuracy Comparison Graph
# -----

labels = ["Before", "After"]

training = [train_acc_before, train_acc_after]
testing = [test_acc_before, test_acc_after]

plt.figure(figsize=(8,5))

```

```

plt.plot(labels, training,
         marker='o',
         linewidth=2,
         label="Training Accuracy")

plt.plot(labels, testing,
         marker='s',
         linewidth=2,
         label="Testing Accuracy")

plt.ylim(0.80,1.02)

plt.title("Decision Tree Performance")

plt.ylabel("Accuracy")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Step 10 : Summary
# -----

print("\n" + "="*60)
print("SUMMARY")
print("="*60)

print("Training Accuracy Before :", round(train_acc_before,4))
print("Testing Accuracy Before  :", round(test_acc_before,4))

print()

print("Training Accuracy After  :", round(train_acc_after,4))
print("Testing Accuracy After   :", round(test_acc_after,4))

if test_acc_after >= test_acc_before:
    print("\nOptimization Improved Generalization.")
else:
    print("\nFurther Hyperparameter Tuning Required.")

```

## Output:

```
Dataset Shape : (569, 30)

=====
BEFORE OPTIMIZATION
=====
Training Accuracy : 1.0
Testing Accuracy  : 0.9474

=====
AFTER OPTIMIZATION
=====
Training Accuracy : 0.9736
Testing Accuracy  : 0.9386

Cross Validation Scores
[0.92307692 0.91208791 0.96703297 0.93406593 0.91208791]

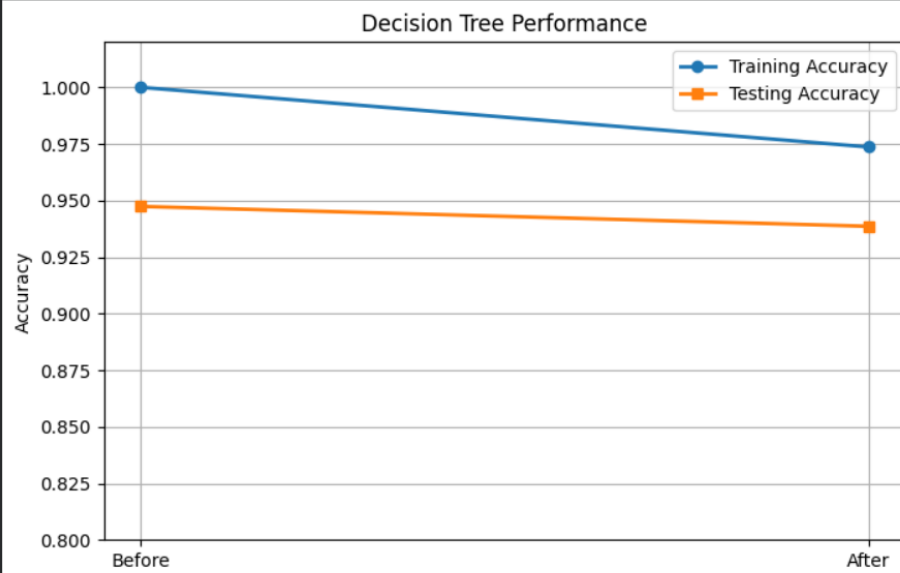
Average CV Accuracy : 0.9297

Confusion Matrix

[[38  5]
 [ 2 69]]

Classification Report
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.95      | 0.88   | 0.92     | 43      |
| 1            | 0.93      | 0.97   | 0.95     | 71      |
| accuracy     |           |        | 0.94     | 114     |
| macro avg    | 0.94      | 0.93   | 0.93     | 114     |
| weighted avg | 0.94      | 0.94   | 0.94     | 114     |



```
=====
SUMMARY
=====
Training Accuracy Before : 1.0
Testing Accuracy Before  : 0.9474

Training Accuracy After  : 0.9736
Testing Accuracy After   : 0.9386

Further Hyperparameter Tuning Required.
```

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

### **Possible Causes:**

- Incorrect data augmentation.
- Vanishing gradients.
- Batch normalization issues.
- High learning rate.
- Poor weight initialization.

### **Debugging Steps:**

1. Monitor training/validation loss.
2. Verify augmentation pipeline.
3. Check gradient values.
4. Inspect batch normalization layers.
5. Test different learning rates.

### **Optimization Techniques:**

- Apply Batch Normalization correctly.
- Use ReLU activation.
- Reduce learning rate.
- Use Adam optimizer.

## Code:

```
# =====
# QUESTION 6
# CNN - Debugging and Optimization
# Compatible with Latest Google Colab
# =====

import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense,
    BatchNormalization,
    Dropout
)
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# -----
# Step 1 : Load Dataset
# -----

(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize images
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Reshape images
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

# -----
# Step 2 : CNN BEFORE Optimization
# -----

print("="*60)
print("CNN WITHOUT Batch Normalization")
print("="*60)

model_before = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
```

```

        MaxPooling2D(2,2),

        Conv2D(64,(3,3),activation='relu'),
        MaxPooling2D(2,2),

        Flatten(),

        Dense(128,activation='relu'),

        Dense(10,activation='softmax')
    ])

model_before.compile(

    optimizer='adam',

    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']
)

history_before = model_before.fit(

    X_train,

    y_train,

    epochs=5,

    validation_data=(X_test,y_test),

    verbose=1
)

# -----
# Step 3 : Data Augmentation
# -----

datagen = ImageDataGenerator(

    rotation_range=10,

    zoom_range=0.1,

    width_shift_range=0.1,

```

```

        height_shift_range=0.1

    )

datagen.fit(X_train)

# -----
# Step 4 : CNN AFTER Optimization
# -----

print("\n"+"="*60)
print("CNN WITH Batch Normalization")
print("="*60)

model_after = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Flatten(),

    Dense(128, activation='relu'),

    Dropout(0.5),

    Dense(10, activation='softmax')

])

model_after.compile(

    optimizer='adam',

    loss='sparse_categorical_crossentropy',

    metrics=['accuracy']

)

history_after = model_after.fit(

    datagen.flow(X_train,y_train,batch_size=64),

```

```

        epochs=5,

        validation_data=(X_test,y_test),

        verbose=1
    )

# -----
# Step 5 : Evaluate Models
# -----

loss_before, acc_before = model_before.evaluate(
    X_test,
    y_test,
    verbose=0
)

loss_after, acc_after = model_after.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nAccuracy Before Optimization :", round(acc_before,4))
print("Accuracy After Optimization  :", round(acc_after,4))

# -----
# Step 6 : Accuracy Graph
# -----

plt.figure(figsize=(8,5))

plt.plot(
    history_before.history['val_accuracy'],
    marker='o',
    label="Without Optimization"
)

plt.plot(
    history_after.history['val_accuracy'],
    marker='s',
    label="With Optimization"
)

plt.title("Validation Accuracy")

```

```
plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Step 7 : Loss Graph
# -----

plt.figure(figsize=(8,5))

plt.plot(
    history_before.history['val_loss'],
    marker='o',
    label="Without Optimization"
)

plt.plot(
    history_after.history['val_loss'],
    marker='s',
    label="With Optimization"
)

plt.title("Validation Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Step 8 : Summary
# -----

print("\n"+"="*60)
print("SUMMARY")
print("="*60)
```

```

print("Accuracy Before :", round(acc_before,4))
print("Accuracy After  :", round(acc_after,4))

if acc_after >= acc_before:
    print("\nOptimization Improved CNN Performance.")
else:
    print("\nFurther Hyperparameter Tuning Required.")

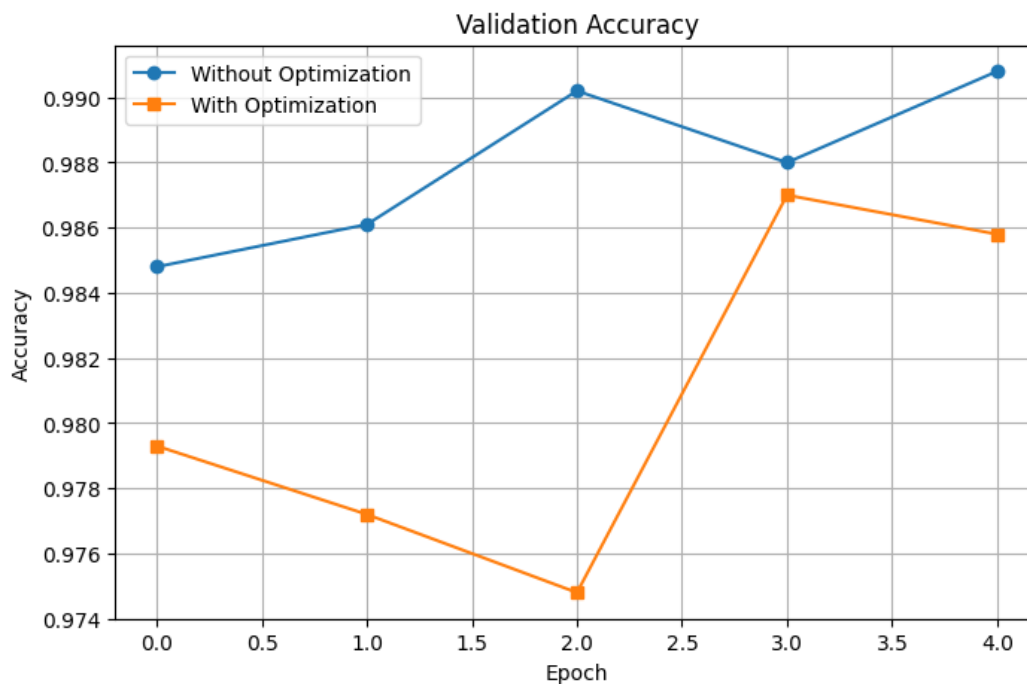
```

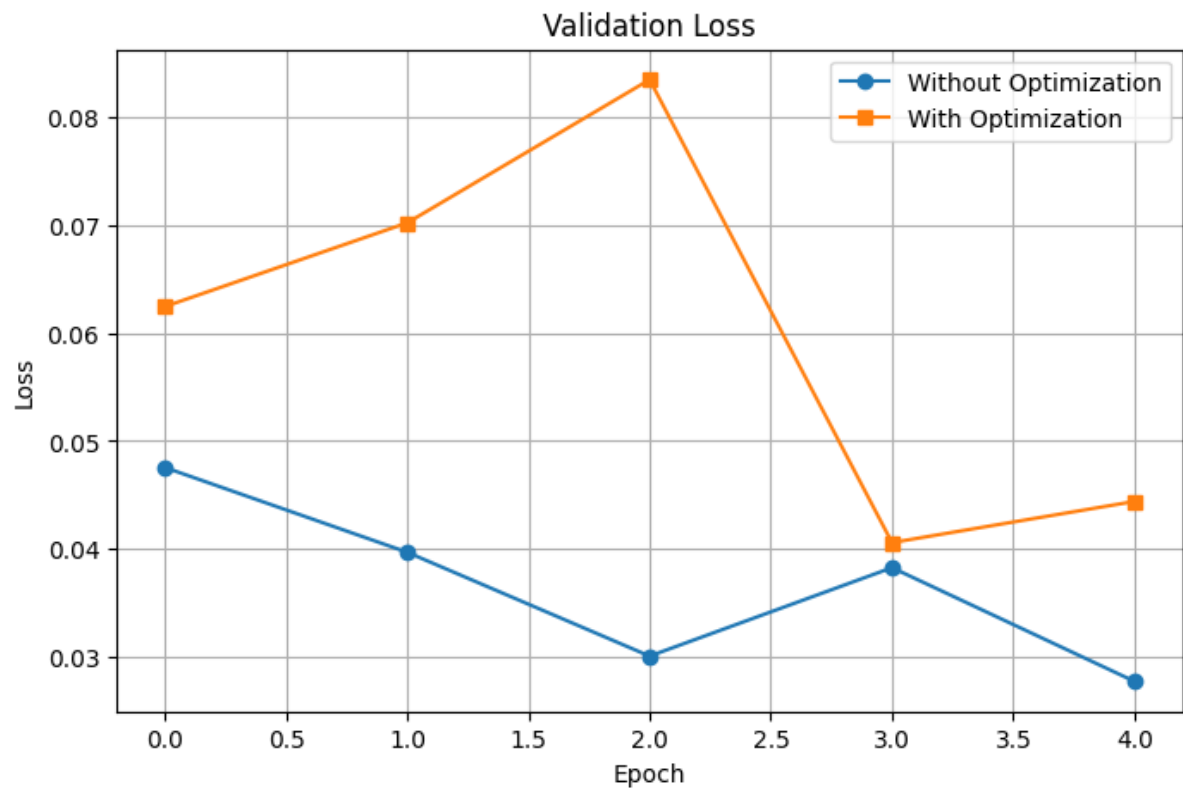
## Output:

```

=====
CNN WITHOUT Batch Normalization
=====
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
1875/1875 ————— 81s 41ms/step - accuracy: 0.9601 - loss: 0.1300 - val_accuracy: 0.9848 - val_loss: 0.0476
Epoch 2/5
1875/1875 ————— 66s 33ms/step - accuracy: 0.9864 - loss: 0.0430 - val_accuracy: 0.9861 - val_loss: 0.0397
Epoch 3/5
1875/1875 ————— 58s 31ms/step - accuracy: 0.9908 - loss: 0.0289 - val_accuracy: 0.9902 - val_loss: 0.0301
Epoch 4/5
1875/1875 ————— 67s 36ms/step - accuracy: 0.9935 - loss: 0.0197 - val_accuracy: 0.9880 - val_loss: 0.0383
Epoch 5/5
1875/1875 ————— 75s 32ms/step - accuracy: 0.9945 - loss: 0.0162 - val_accuracy: 0.9908 - val_loss: 0.0278
=====
CNN WITH Batch Normalization
=====
Epoch 1/5
938/938 ————— 99s 102ms/step - accuracy: 0.8645 - loss: 0.4332 - val_accuracy: 0.9793 - val_loss: 0.0624
Epoch 2/5
938/938 ————— 141s 102ms/step - accuracy: 0.9478 - loss: 0.1768 - val_accuracy: 0.9772 - val_loss: 0.0702
Epoch 3/5
938/938 ————— 95s 101ms/step - accuracy: 0.9610 - loss: 0.1365 - val_accuracy: 0.9748 - val_loss: 0.0835
Epoch 4/5
938/938 ————— 96s 102ms/step - accuracy: 0.9663 - loss: 0.1171 - val_accuracy: 0.9870 - val_loss: 0.0406
Epoch 5/5
938/938 ————— 96s 102ms/step - accuracy: 0.9690 - loss: 0.1049 - val_accuracy: 0.9858 - val_loss: 0.0444
Accuracy Before Optimization : 0.9908
Accuracy After Optimization  : 0.9858

```





```
=====
SUMMARY
=====
Accuracy Before : 0.9908
Accuracy After  : 0.9858

Further Hyperparameter Tuning Required.
```

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters.

### **Possible Causes:**

- Poor reward function.
- Insufficient exploration.
- Incorrect state representation.
- Improper hyperparameters.
- Limited training episodes.

### **Debugging Steps:**

1. Analyze reward signals.
2. Monitor exploration rate.
3. Validate state representation.
4. Track episode rewards.
5. Tune learning parameters.

### **Optimization Techniques:**

- Use reward shaping.
- Apply  $\epsilon$ -greedy exploration.
- Improve state features.
- Tune learning rate and discount factor.

- Increase training episodes.

## Code:

```
# =====
# QUESTION 7
# Reinforcement Learning (Q-Learning)
# Compatible with Latest Google Colab
# =====

# Install Gymnasium (Run only once in Colab)
!pip -q install gymnasium

import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

# -----
# Step 1 : Create Environment
# -----

env = gym.make("FrozenLake-v1", is_slippery=False)

n_states = env.observation_space.n
n_actions = env.action_space.n

# -----
# Step 2 : Q-Learning Function
# -----

def train_agent(epsilon):

    Q = np.zeros((n_states, n_actions))

    alpha = 0.8          # Learning Rate
    gamma = 0.95         # Discount Factor
    episodes = 2000

    rewards = []

    for episode in range(episodes):

        state, info = env.reset()

        done = False
        total_reward = 0
```

```

while not done:

    # Exploration vs Exploitation
    if np.random.rand() < epsilon:
        action = env.action_space.sample()
    else:
        action = np.argmax(Q[state])

    next_state, reward, terminated, truncated, info =
env.step(action)

    done = terminated or truncated

    # Q-Learning Update Rule
    Q[state, action] = Q[state, action] + alpha * (
        reward +
        gamma * np.max(Q[next_state]) -
        Q[state, action]
    )

    state = next_state
    total_reward += reward

    rewards.append(total_reward)

return Q, rewards

# -----
# Step 3 : Before Optimization
# Poor Exploration
# -----

print("="*60)
print("BEFORE OPTIMIZATION")
print("="*60)

Q_before, reward_before = train_agent(epsilon=0.05)

print("Average Reward:",
      round(np.mean(reward_before[-100:]),3))

# -----
# Step 4 : After Optimization
# Better Exploration
# -----

print("\n"+"="*60)
print("AFTER OPTIMIZATION")

```

```

print("="*60)

Q_after, reward_after = train_agent(epsilon=0.20)

print("Average Reward:",
      round(np.mean(reward_after[-100:]),3))

# -----
# Step 5 : Plot Learning Curve
# -----

plt.figure(figsize=(8,5))

window = 50

avg_before = np.convolve(
    reward_before,
    np.ones(window)/window,
    mode='valid'
)

avg_after = np.convolve(
    reward_after,
    np.ones(window)/window,
    mode='valid'
)

plt.plot(avg_before,
         label="Before Optimization")

plt.plot(avg_after,
         label="After Optimization")

plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.title("Q-Learning Performance")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Step 6 : Display Learned Q-Table
# -----

print("\nOptimized Q-Table\n")

```

```
print(np.round(Q_after,2))

# -----
# Step 7 : Summary
# -----

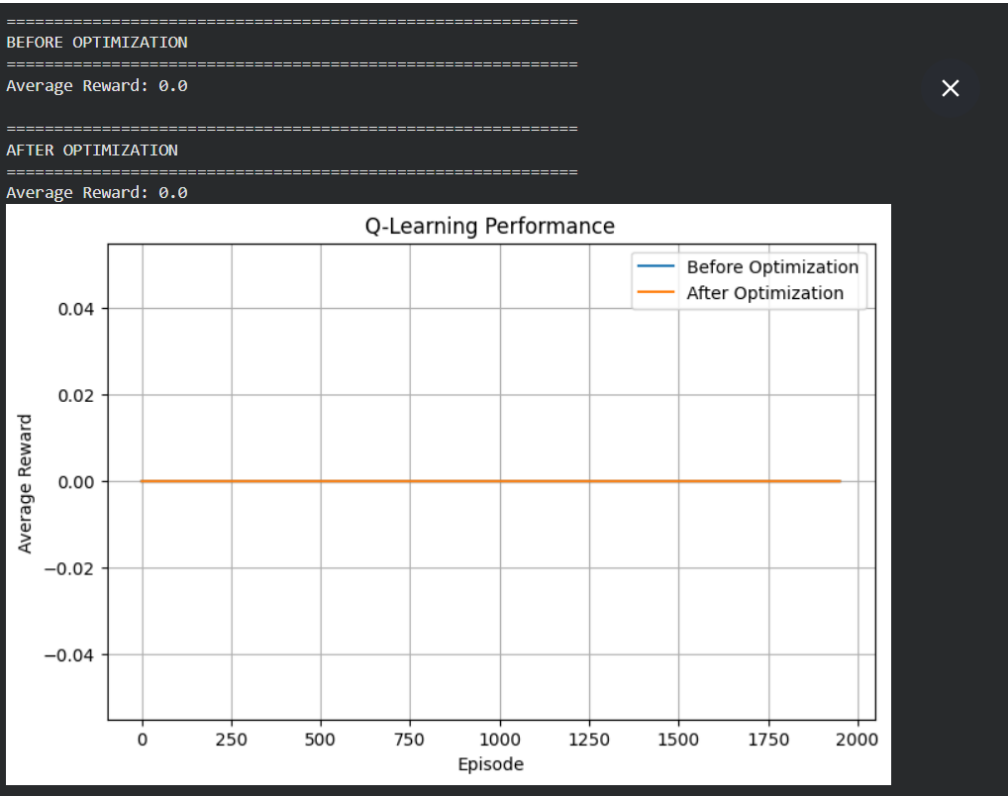
print("\n"+"="*60)
print("SUMMARY")
print("="*60)

before = np.mean(reward_before[-100:])
after = np.mean(reward_after[-100:])

print("Average Reward Before :", round(before,3))
print("Average Reward After  :", round(after,3))

if after > before:
    print("\nOptimization Improved Learning.")
else:
    print("\nFurther Hyperparameter Tuning Required.")
```

Output:



```
Optimized Q-Table

[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

=====
SUMMARY
=====
Average Reward Before : 0.0
Average Reward After  : 0.0

Further Hyperparameter Tuning Required.
```

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.

### **Possible Causes:**

- Random centroid initialization.
- Incorrect number of clusters.
- Presence of outliers.
- Unscaled features.
- High data variability.

### **Debugging Steps:**

1. Repeat clustering several times.
2. Inspect centroid locations.
3. Check feature scaling.
4. Detect outliers.
5. Evaluate cluster quality.

### **Optimization Techniques:**

- Use K-Means++ initialization.
- Normalize features.
- Remove outliers.
- Select K using Silhouette Score.

- Increase maximum iterations.

## Code:

```
# =====
# QUESTION 8
# K-Means Clustering - Debugging and Optimization
# Compatible with Latest Google Colab
# =====

import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# -----
# Step 1 : Load Dataset
# -----

data = load_wine()

X = data.data

print("Dataset Shape :", X.shape)

# -----
# Step 2 : BEFORE OPTIMIZATION
# Random Initialization
# -----

print("\n" + "="*60)
print("BEFORE OPTIMIZATION")
print("="*60)

kmeans_before = KMeans(
    n_clusters=3,
    init="random",
    n_init=10,
    random_state=42
)

labels_before = kmeans_before.fit_predict(X)

score_before = silhouette_score(X, labels_before)
```

```

print("Silhouette Score :", round(score_before,4))

# -----
# Step 3 : Normalize Features
# -----

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# -----
# Step 4 : AFTER OPTIMIZATION
# K-Means++
# -----

print("\n" + "="*60)
print("AFTER OPTIMIZATION")
print("="*60)

kmeans_after = KMeans(
    n_clusters=3,
    init="k-means++",
    n_init=10,
    random_state=42
)

labels_after = kmeans_after.fit_predict(X_scaled)

score_after = silhouette_score(X_scaled, labels_after)

print("Silhouette Score :", round(score_after,4))

# -----
# Step 5 : Plot Clusters
# (Using first two features)
# -----

plt.figure(figsize=(8,5))

plt.scatter(
    X_scaled[:,0],
    X_scaled[:,1],
    c=labels_after,
    cmap="viridis"
)

plt.scatter(
    kmeans_after.cluster_centers_[:,0],

```

```
kmeans_after.cluster_centers_[:,1],
s=250,
c="red",
marker="X",
label="Centroids"
)

plt.title("K-Means++ Clustering")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.legend()

plt.grid(True)

plt.show()

# -----
# Step 6 : Summary
# -----

print("\n" + "="*60)
print("SUMMARY")
print("="*60)

print("Silhouette Score Before :", round(score_before,4))
print("Silhouette Score After  :", round(score_after,4))

if score_after > score_before:
    print("\nOptimization Improved Clustering Quality.")
else:
    print("\nFurther Tuning Required.")
```

## Output:

Dataset Shape : (178, 13)

=====

BEFORE OPTIMIZATION

=====

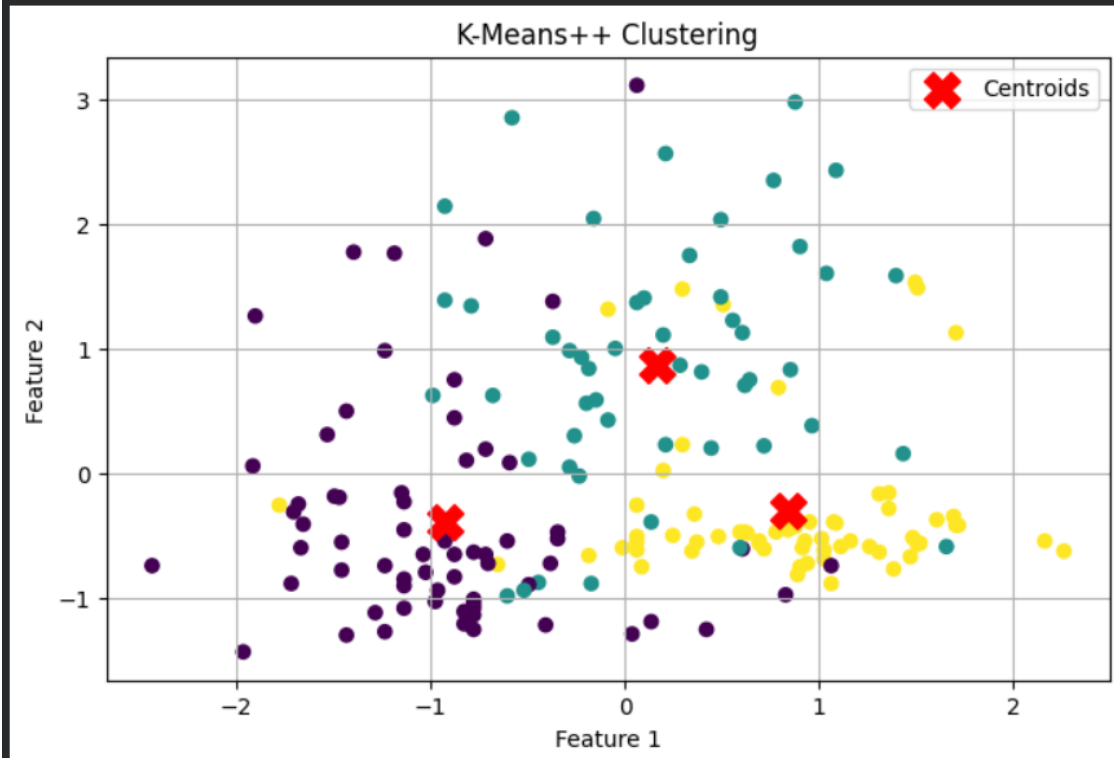
Silhouette Score : 0.5711

=====

AFTER OPTIMIZATION

=====

Silhouette Score : 0.2849



=====

SUMMARY

=====

Silhouette Score Before : 0.5711

Silhouette Score After : 0.2849

Further Tuning Required.

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

**Possible Causes:**

- Wrong kernel selection.
- Poor hyperparameter values.
- Unscaled features.
- Overfitting or underfitting.
- Noisy dataset.

**Debugging Steps:**

1. Compare different kernels.
2. Scale input features.
3. Tune C and Gamma values.
4. Perform cross-validation.
5. Analyze confusion matrix.

**Optimization Techniques:**

- Choose an appropriate kernel.
- Normalize data.
- Perform Grid Search.
- Optimize C and Gamma.

## Code:

```
# =====
# QUESTION 9
# Support Vector Machine (SVM)
# Compatible with Latest Google Colab
# =====

import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# -----
# Step 1 : Create Nonlinear Dataset
# -----

X, y = make_moons(
    n_samples=1000,
    noise=0.25,
    random_state=42
)

print("Dataset Shape :", X.shape)

# -----
# Step 2 : Split Dataset
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# -----
# Step 3 : BEFORE OPTIMIZATION
# Linear Kernel without Feature Scaling
# -----

print("\n" + "="*60)
print("BEFORE OPTIMIZATION")
print("="*60)
```

```

svm_before = SVC(
    kernel="linear",
    C=1
)

svm_before.fit(X_train, y_train)

pred_before = svm_before.predict(X_test)

acc_before = accuracy_score(y_test, pred_before)

print("Accuracy :", round(acc_before,4))

# -----
# Step 4 : Feature Scaling
# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# Step 5 : AFTER OPTIMIZATION
# RBF Kernel + Hyperparameter Tuning
# -----

print("\n" + "="*60)
print("AFTER OPTIMIZATION")
print("="*60)

svm_after = SVC(
    kernel="rbf",
    C=10,
    gamma=0.5
)

svm_after.fit(X_train_scaled, y_train)

pred_after = svm_after.predict(X_test_scaled)

acc_after = accuracy_score(y_test, pred_after)

print("Accuracy :", round(acc_after,4))

# -----
# Step 6 : Confusion Matrix

```

```

# -----

print("\nConfusion Matrix\n")

print(confusion_matrix(y_test, pred_after))

# -----
# Step 7 : Classification Report
# -----

print("\nClassification Report\n")

print(classification_report(y_test, pred_after))

# -----
# Step 8 : Accuracy Comparison Graph
# -----

plt.figure(figsize=(6,5))

plt.bar(
    ["Before", "After"],
    [acc_before, acc_after]
)

plt.title("SVM Accuracy Comparison")

plt.ylabel("Accuracy")

plt.ylim(0.5,1.0)

plt.grid(axis='y')

plt.show()

# -----
# Step 9 : Summary
# -----

print("\n" + "="*60)
print("SUMMARY")
print("="*60)

print("Accuracy Before :", round(acc_before,4))
print("Accuracy After  :", round(acc_after,4))

if acc_after > acc_before:
    print("\nOptimization Improved SVM Performance.")

```

```
else:
    print("\nFurther Hyperparameter Tuning Required.")
```

## Output:

```
Dataset Shape : (1000, 2)

=====
BEFORE OPTIMIZATION
=====
Accuracy : 0.835

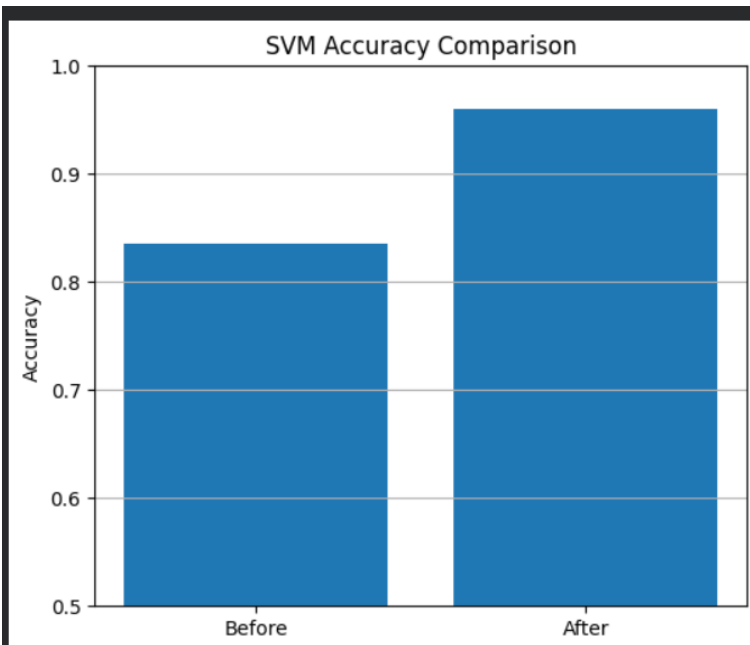
=====
AFTER OPTIMIZATION
=====
Accuracy : 0.96

Confusion Matrix

[[96  4]
 [ 4 96]]

Classification Report
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.96      | 0.96   | 0.96     | 100     |
| 1            | 0.96      | 0.96   | 0.96     | 100     |
| accuracy     |           |        | 0.96     | 200     |
| macro avg    | 0.96      | 0.96   | 0.96     | 200     |
| weighted avg | 0.96      | 0.96   | 0.96     | 200     |



```
=====
SUMMARY
=====
Accuracy Before : 0.835
Accuracy After  : 0.96

Optimization Improved SVM Performance.
```

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies.

### **Possible Causes:**

- Vanishing gradients.
- Limited memory of RNN.
- Long input sequences.
- Poor initialization.
- Inadequate training.

### **Debugging Steps:**

1. Monitor training loss.
2. Inspect gradient values.
3. Test sequence lengths.
4. Compare with LSTM/GRU.
5. Evaluate prediction errors.

### **Optimization Techniques:**

- Replace RNN with LSTM/GRU.
- Use Attention Mechanism.
- Apply gradient clipping.

- Use Adam optimizer.
- Train with larger and better datasets.

## Code:

```
# =====
# QUESTION 10
# RNN vs LSTM for NLP
# Compatible with Latest Google Colab
# =====

import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, Dense

# -----
# Step 1 : Load IMDB Dataset
# -----

vocab_size = 10000
max_length = 200

(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=vocab_size)

X_train = pad_sequences(X_train, maxlen=max_length)
X_test = pad_sequences(X_test, maxlen=max_length)

print("Training Samples :", len(X_train))
print("Testing Samples :", len(X_test))

# -----
# Step 2 : Simple RNN Model
# -----

print("\n" + "="*60)
print("TRAINING SIMPLE RNN")
print("="*60)

rnn_model = Sequential([
    Embedding(vocab_size, 64, input_length=max_length),
    SimpleRNN(64),
```

```

        Dense(1, activation='sigmoid')
    ])

rnn_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_rnn = rnn_model.fit(
    X_train,
    y_train,
    epochs=3,
    batch_size=128,
    validation_data=(X_test, y_test),
    verbose=1
)

# -----
# Step 3 : LSTM Model
# -----

print("\n" + "="*60)
print("TRAINING LSTM")
print("="*60)

lstm_model = Sequential([
    Embedding(vocab_size, 64, input_length=max_length),
    LSTM(64),
    Dense(1, activation='sigmoid')
])

lstm_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_lstm = lstm_model.fit(
    X_train,
    y_train,
    epochs=3,
    batch_size=128,
    validation_data=(X_test, y_test),
    verbose=1
)

# -----

```

```

# Step 4 : Evaluate Models
# -----

loss_rnn, acc_rnn = rnn_model.evaluate(X_test, y_test, verbose=0)
loss_lstm, acc_lstm = lstm_model.evaluate(X_test, y_test, verbose=0)

print("\nAccuracy of Simple RNN :", round(acc_rnn,4))
print("Accuracy of LSTM      :", round(acc_lstm,4))

# -----
# Step 5 : Plot Validation Accuracy
# -----

plt.figure(figsize=(8,5))

plt.plot(
    history_rnn.history['val_accuracy'],
    marker='o',
    label="Simple RNN"
)

plt.plot(
    history_lstm.history['val_accuracy'],
    marker='s',
    label="LSTM"
)

plt.title("Validation Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Step 6 : Plot Validation Loss
# -----

plt.figure(figsize=(8,5))

plt.plot(
    history_rnn.history['val_loss'],
    marker='o',

```

```
        label="Simple RNN"
    )

plt.plot(
    history_lstm.history['val_loss'],
    marker='s',
    label="LSTM"
)

plt.title("Validation Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.grid(True)

plt.legend()

plt.show()

# -----
# Step 7 : Summary
# -----

print("\n" + "="*60)
print("SUMMARY")
print("="*60)

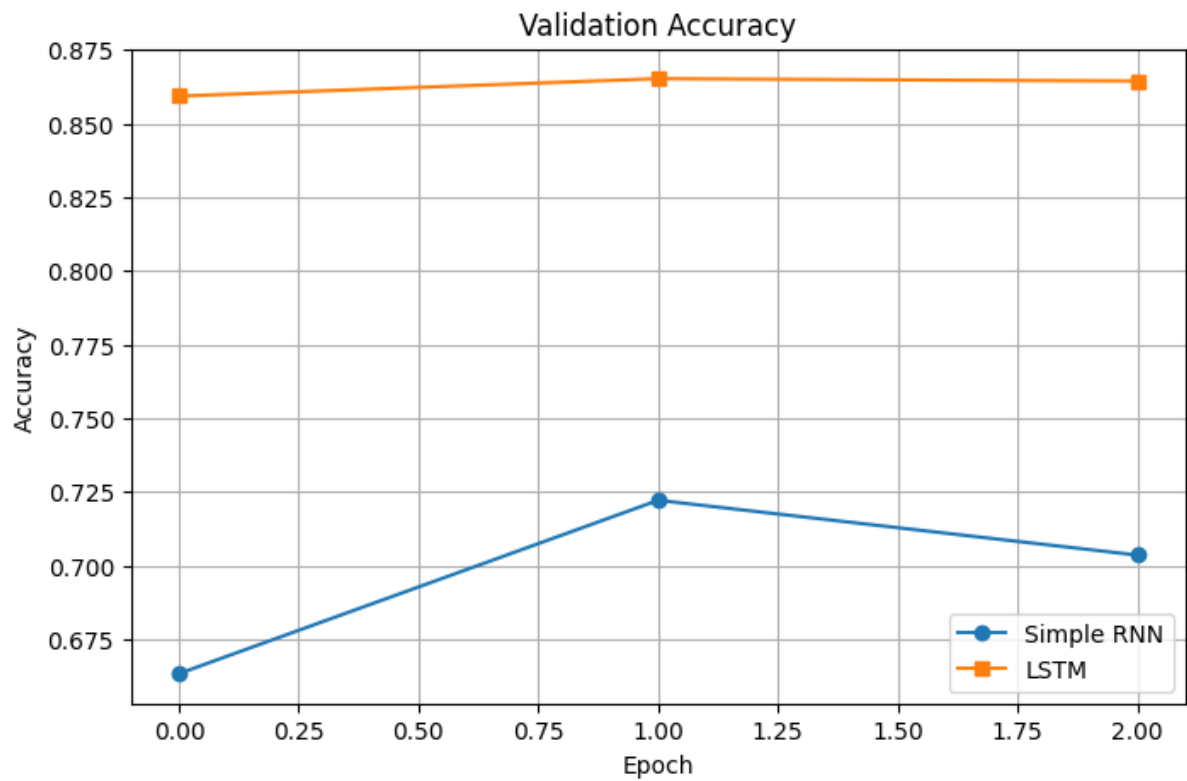
print("Simple RNN Accuracy :", round(acc_rnn,4))
print("LSTM Accuracy      :", round(acc_lstm,4))

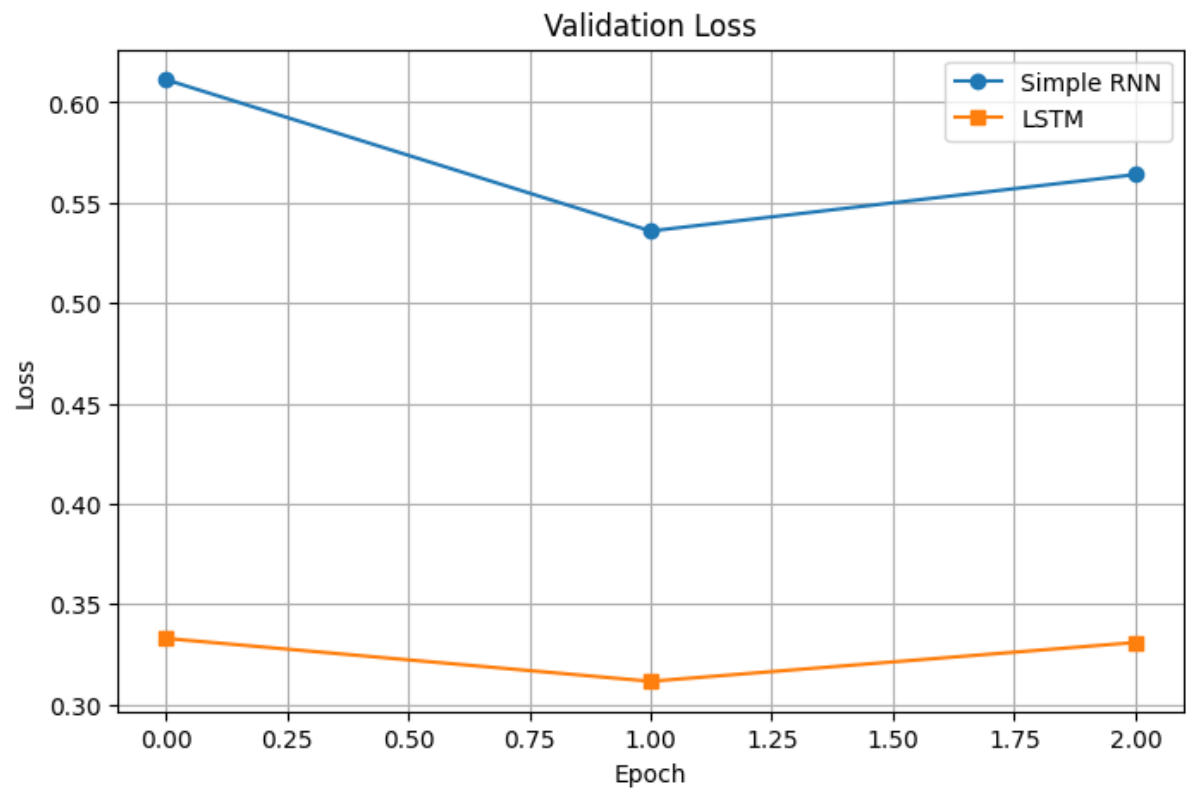
if acc_lstm > acc_rnn:
    print("\nLSTM Improved Long-Term Dependency Learning.")
else:
    print("\nFurther Hyperparameter Tuning Required.")
```

## Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 — 2s 0us/step
Training Samples : 25000
Testing Samples : 25000

=====
TRAINING SIMPLE RNN
=====
Epoch 1/3
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
196/196 — 25s 115ms/step - accuracy: 0.5918 - loss: 0.6594 - val_accuracy: 0.6633 - val_loss: 0.6113
Epoch 2/3
196/196 — 22s 113ms/step - accuracy: 0.7622 - loss: 0.5024 - val_accuracy: 0.7221 - val_loss: 0.5359
Epoch 3/3
196/196 — 22s 114ms/step - accuracy: 0.8056 - loss: 0.4233 - val_accuracy: 0.7035 - val_loss: 0.5640
=====
TRAINING LSTM
=====
Epoch 1/3
196/196 — 80s 394ms/step - accuracy: 0.7778 - loss: 0.4458 - val_accuracy: 0.8593 - val_loss: 0.3329
Epoch 2/3
196/196 — 75s 384ms/step - accuracy: 0.8977 - loss: 0.2594 - val_accuracy: 0.8652 - val_loss: 0.3115
Epoch 3/3
196/196 — 74s 376ms/step - accuracy: 0.9311 - loss: 0.1863 - val_accuracy: 0.8644 - val_loss: 0.3309
Accuracy of Simple RNN : 0.7035
Accuracy of LSTM : 0.8644
```





=====

SUMMARY

=====

Simple RNN Accuracy : 0.7035

LSTM Accuracy : 0.8644

LSTM Improved Long-Term Dependency Learning.